

Java 8, 11 & 17 — Biggest Updates

Focus on the features that changed everyday Java development and the ones interviewers commonly ask.

The simplest view:

```
Java 8
→ Functional programming + Streams + Optional + Date/Time + default methods

Java 11
→ Modern standard APIs + HTTP Client + var in lambdas + runtime improvements

Java 17
→ Records + Sealed Classes + Pattern Matching improvements + stronger modern Java baseline
```

All three are **LTS releases**:

```
Java 8   → LTS
Java 11  → LTS
Java 17  → LTS
```

1. Java 8 — The Biggest Change

Java 8 was arguably the biggest change to day-to-day Java programming because it introduced a more functional programming style.

The most important features:

```
Java 8
├─ Lambda expressions
├─ Stream API
├─ Functional interfaces
├─ Method references
├─ Optional
├─ Default/static methods in interfaces
├─ java.time API
└─ CompletableFuture
```

2. Lambda Expressions

Before Java 8:

```
List<String> names = Arrays.asList(
    "Avnish", "Rahul", "Amit");

Collections.sort(names, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
});
```

Java 8:

```
names.sort((a, b) -> a.compareTo(b));
```

Or:

```
names.sort(String::compareTo);
```

A lambda lets you represent a piece of behavior/function without creating an anonymous class.

Interview definition

A lambda expression is a concise way to represent an implementation of a functional interface.

3. Functional Interfaces

A functional interface has exactly **one abstract method**.

Example:

```
@FunctionalInterface
interface PaymentValidator {
    boolean validate(Payment payment);
}
```

Then:

```
PaymentValidator validator =
    payment -> payment.getAmount().signum() > 0;
```

Important built-in functional interfaces:

```
Predicate<T>   → T → boolean
Function<T,R>  → T → R
Consumer<T>    → T → void
Supplier<T>    → () → T
```

Example:

```
Predicate<Payment> valid =
    p -> p.getAmount().signum() > 0;
```

4. Stream API

One of the most important Java 8 interview topics.

Before:

```
List<Payment> paidPayments = new ArrayList<>();

for (Payment p : payments) {
    if ("PAID".equals(p.getStatus())) {
        paidPayments.add(p);
    }
}
```

Java 8:

```
List<Payment> paidPayments =
    payments.stream()
        .filter(p -> "PAID".equals(p.getStatus()))
        .toList();
```

Typical operations:

```
filter
map
flatMap
sorted
distinct
limit
skip
reduce
collect
groupingBy
partitioningBy
```

5. Stream Pipeline

Think:

```
Source
↓
Intermediate operations
↓
Terminal operation
```

Example:

```
payments.stream()
    .filter(p -> "PAID".equals(p.getStatus()))
    .map(Payment::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

```
payments
↓
filter
↓
map
↓
reduce
```

Important interview concept:

Intermediate operations are generally lazy; execution happens when a terminal operation is invoked.

6. `map()` VS `flatMap()`

Very common interview question.

`map`

Transforms one object into one object:

```
List<String> statuses =
    payments.stream()
        .map(Payment::getStatus)
        .toList();
```

Conceptually:

```
Payment → String
```

flatMap

Flattens nested structures.

Suppose:

```
List<Customer> customers;
```

and each customer has:

```
List<Payment>
```

Then:

```
List<Payment> payments =  
    customers.stream()  
        .flatMap(c -> c.getPayments().stream())  
        .toList();
```

Conceptually:

```
List<List<Payment>>  
    ↓  
flatMap  
    ↓  
List<Payment>
```

7. Optional

Java 8 introduced `Optional<T>` as a way to represent an optional value.

Instead of:

```
Payment payment = repository.find(id);  
  
if (payment != null) {  
    ...  
}
```

you might have:

```
Optional<Payment> payment =  
    repository.findById(id);
```

Then:

```
Payment payment = repository.findById(id)  
    .orElseThrow(() ->  
        new PaymentNotFoundException("Not found"));
```

Important methods:

```
of()  
ofNullable()  
empty()  
isPresent()  
ifPresent()  
map()
```

```
orElse()  
orElseGet()  
orElseThrow()
```

8. `orElse()` VS `orElseGet()`

Classic interview question.

```
optional.orElse(expensiveOperation());
```

The argument can be evaluated even if the value exists.

Whereas:

```
optional.orElseGet(() -> expensiveOperation());
```

evaluates the supplier only when the value is absent.

So:

```
orElse    → eager argument evaluation  
orElseGet → lazy supplier evaluation
```

9. Default Methods in Interfaces

Java 8 allowed interfaces to contain default implementations.

```
interface PaymentProcessor {  
    void process();  
  
    default void log() {  
        System.out.println("Payment processing");  
    }  
}
```

This helped Java evolve interfaces without immediately forcing every existing implementation to implement a newly added method.

Interfaces can also contain static methods.

10. `java.time` API

Java 8 introduced the modern date/time API.

Old:

```
Date  
Calendar
```

Modern:

```
LocalDate  
LocalDateTime  
LocalTime  
Instant  
ZonedDateTime
```

Duration
Period

Example:

```
LocalDate today = LocalDate.now();

LocalDate expiry =
    today.plusDays(30);
```

For backend systems, know:

```
LocalDate
→ date without time zone

Instant
→ point on UTC timeline

ZonedDateTime
→ date/time with zone
```

This is much safer and clearer than relying heavily on the old mutable `Date` / `Calendar` APIs.

11. CompletableFuture

Java 8 also introduced a much better model for composing asynchronous operations.

```
CompletableFuture
    .supplyAsync(() -> fetchPayment())
    .thenApply(this::validate)
    .thenApply(this::calculateAmount)
    .thenAccept(this::save);
```

Important methods:

```
supplyAsync
runAsync
thenApply
thenAccept
thenCompose
thenCombine
exceptionally
handle
allOf
anyOf
```

This becomes relevant when discussing:

- async API calls
- parallel independent tasks
- non-blocking workflows

12. Java 8 Interview Must-Know

For Java 8, prioritize:

```
***** Lambda
***** Stream API
***** Functional Interfaces
***** Optional
***** Method References
```

```
★★★★☆ CompletableFuture
★★★★☆ java.time
★★★★☆ Default Methods
```

13. Java 11 — What Actually Changed?

Java 11 is less revolutionary than Java 8, but it introduced several useful language/library/runtime improvements.

Important ones:

```
Java 11
├─ Standard HTTP Client
├─ String API enhancements
├─ Files API enhancements
├─ `var` in lambda parameters
├─ Single-file source execution
├─ NESTMATE access improvements
└─ JVM/runtime improvements
```

14. Java 11 HTTP Client

One of the biggest practical additions.

Before Java 11, developers often used:

```
URLConnection
Apache HttpClient
Spring RestTemplate
```

Java 11 introduced:

```
java.net.http.HttpClient
```

Example:

```
HttpClient client =
    HttpClient.newHttpClient();

HttpRequest request =
    HttpRequest.newBuilder()
        .uri(URI.create(
            "https://example.com"))
        .GET()
        .build();

HttpResponse<String> response =
    client.send(
        request,
        HttpResponse.BodyHandlers.ofString()
    );
```

It supports synchronous and asynchronous requests.

Async:

```
client.sendAsync(
    request,
    HttpResponse.BodyHandlers.ofString()
);
```

For your interview:

Java 11 introduced the standardized modern HTTP Client API with HTTP/2 and asynchronous support.

In a Spring Boot project, however, you may still encounter `RestTemplate`, `WebClient`, Feign, Apache HttpClient, etc., depending on the architecture.

15. String API Improvements

Java 11 added useful methods:

```
" ".isBlank();

"hello\nworld".lines();

" hello ".strip();

"abc ".stripTrailing();

" abc".stripLeading();

"ha".repeat(3);
```

Results:

```
isBlank()
lines()
strip()
stripLeading()
stripTrailing()
repeat()
```

Important difference:

```
trim() → older semantics
strip() → Unicode-aware whitespace handling
```

16. Files Improvements

Java 11 added convenient:

```
Files.readString(path);
Files.writeString(path, content);
```

Instead of:

```
Files.readAllBytes(...)
```

and manually converting bytes.

Example:

```
String content =
    Files.readString(Path.of("payment.txt"));
```

17. `var` in Lambda Parameters

Java 10 introduced local variable type inference.

Java 11 extended `var` to lambda parameters.

```
(var a, var b) -> a.compareTo(b)
```

Why would you use this?

Mostly when you need annotations:

```
(@Nullable var payment) -> ...
```

You cannot mix:

```
(var a, b) -> ...
```

You need consistent syntax:

```
(var a, var b) -> ...
```

18. Single-File Source Execution

Java 11 allows:

```
java Hello.java
```

without explicitly compiling first in the traditional workflow.

Useful for:

- quick scripts
- demos
- experiments
- small utilities

Not a major enterprise application feature, but worth knowing.

19. Java 11 Removed/Deprecated Some Things

Some older Java EE modules that had been deprecated in Java 9 were removed from the JDK by Java 11, such as older Java EE/CORBA modules.

This matters when migrating older applications from Java 8 to Java 11.

Also:

```
Java 8 application
↓
Upgrade to Java 11
↓
Check removed JDK modules / dependencies
```

In Spring applications, some libraries may need explicit dependencies after migration.

20. Java 17 — Major Developer-Facing Features

Java 17 is especially important because many enterprises standardized on Java 17.

The interview-relevant features:

```
Java 17
├─ Records
├─ Sealed Classes
├─ Pattern Matching for instanceof
├─ Switch enhancements (preview at 17)
├─ Text Blocks (standardized earlier, but commonly used by 17)
├─ Strong encapsulation of JDK internals
└─ JVM/runtime improvements
```

Some pattern/switch features were still preview features in Java 17, so don't describe all switch pattern matching as fully finalized in 17.

21. Records

One of the biggest Java 17 features.

Before:

```
public final class PaymentResponse {

    private final Long id;
    private final String status;

    public PaymentResponse(
        Long id,
        String status) {
        this.id = id;
        this.status = status;
    }

    public Long getId() {
        return id;
    }

    public String getStatus() {
        return status;
    }

    // equals
    // hashCode
    // toString
}
```

Java 17:

```
public record PaymentResponse(
    Long id,
    String status) {
}
```

The compiler provides the standard record semantics for:

- constructor
- accessors
- equals
- hashCode
- toString

For example:

```
PaymentResponse response =  
    new PaymentResponse(1L, "PAID");  
  
response.id();  
response.status();
```

Records are excellent for:

```
DTOs  
API responses  
immutable data carriers  
small value-like objects
```

22. Records Are Not Just “Immutable Classes”

Be precise.

Records are designed as **transparent carriers for immutable-ish state**, with final component fields and generated methods.

But the referenced objects themselves aren’t magically deeply immutable.

For example:

```
record Order(List<String> items) {}
```

The record component reference is final, but the `List` can still be mutable unless you make the contained object immutable or defensively copy it.

23. Records and JPA

Very important for your backend interviews.

Do **not** simply say:

```
“Replace all JPA entities with records.”
```

JPA entities typically need:

- identity
- entity lifecycle
- proxies/enhancement considerations
- no-arg constructor requirements depending on mapping/provider configuration
- mutable state for normal ORM usage

Records are generally a much better fit for:

```
DTOs  
API request/response models  
projections/value carriers
```

than ordinary JPA entities.

24. Pattern Matching for `instanceof`

Before:

```
if (obj instanceof Payment) {  
    Payment payment = (Payment) obj;  
    System.out.println(payment.getStatus());  
}
```

Java 17:

```
if (obj instanceof Payment payment) {  
    System.out.println(payment.getStatus());  
}
```

The cast variable is introduced directly as part of the `instanceof` check.

Cleaner and safer.

25. Pattern Matching: Why Useful?

Consider:

```
if (obj instanceof String s && !s.isBlank()) {  
    process(s);  
}
```

You avoid:

```
if (obj instanceof String) {  
    String s = (String) obj;  
    ...  
}
```

This becomes increasingly useful as pattern matching evolves in later Java releases.

26. Sealed Classes

Java 17 introduced sealed classes as a standard feature.

Suppose payment types are controlled:

```
public sealed interface Payment  
    permits CardPayment,  
             BankPayment {  
  
}
```

Then:

```
public final class CardPayment  
    implements Payment {  
  
}
```

```
public final class BankPayment  
    implements Payment {  
  
}
```

The key idea:

The parent explicitly controls which classes can extend/implement it.

27. Why Sealed Classes?

Without sealed classes:

```
Payment
↓
any class can implement it
```

With sealed:

```
Payment
├── CardPayment
└── BankPayment
```

The hierarchy is controlled.

Useful when your domain has a **closed set of variants**.

Examples:

```
Payment
├── Card
├── BankTransfer
└── Wallet

Result
├── Success
└── Failure
```

This also enables more exhaustive reasoning with pattern matching.

28. `final`, `sealed`, `non-sealed`

In sealed hierarchies, an allowed subclass can be:

```
final
```

meaning no further extension,

or:

```
sealed
```

meaning it controls its own permitted subclasses,

or:

```
non-sealed
```

meaning the restriction is opened again.

Example:

```
public sealed class Payment
    permits CardPayment, BankPayment {
}
```

29. Java 17 Switch Pattern Matching

Java 17 had **pattern matching for switch as a preview feature**, not a finalized standard feature.

Conceptually:

```
return switch (payment) {  
    case CardPayment c -> processCard(c);  
    case BankPayment b -> processBank(b);  
};
```

This is worth knowing historically, but in an interview don't claim:

"Pattern matching switch was fully finalized in Java 17."

It matured in later Java releases.

30. Text Blocks

Text blocks became standard before Java 17, but they're commonly encountered in Java 17 codebases.

Instead of:

```
String json =  
    "{\n" +  
    "  \"status\": \"PAID\"\n" +  
    "}";
```

you can write:

```
String json = ""  
    {  
        "status": "PAID"  
    }  
    "";
```

Very useful for:

- JSON
- SQL
- HTML
- multi-line strings

And you'll frequently see it in modern Spring code:

```
@Query("""  
    SELECT p  
    FROM Payment p  
    WHERE p.status = :status  
    """)
```

31. Strong Encapsulation

Java 17 strengthened encapsulation of internal JDK APIs as part of the broader Java platform evolution.

This matters mainly during migrations of older applications that relied on internal JDK implementation

details.

For enterprise migration:

```
Java 8
  ↓
Java 11
  ↓
Java 17
```

you should test libraries that depend on:

- internal JDK APIs
- reflection against JDK internals
- removed modules
- old bytecode tooling

32. Java 8 vs 11 vs 17

Feature	Java 8	Java 11	Java 17
Lambda	✓	✓	✓
Stream API	✓	✓	✓
Optional	✓	✓	✓
Functional interfaces	✓	✓	✓
<code>java.time</code>	✓	✓	✓
<code>CompletableFuture</code>	✓	✓	✓
Modern HTTP Client	✗	✓	✓
<code>String.isBlank()</code>	✗	✓	✓
<code>Files.readString()</code>	✗	✓	✓
<code>var</code> local variables	✗	✓	✓
<code>var</code> in lambda params	✗	✓	✓
Records	✗	✗	✓
Sealed classes	✗	✗	✓
Pattern matching <code>instanceof</code>	✗	✗	✓
Text blocks	✗	Preview	✓
Stronger modern JVM baseline	Older	Improved	Improved

33. The Most Important Interview Distinction

If interviewer asks:

“What major features came in Java 8, 11 and 17?”

A good answer is:

Java 8

Lambda

```
Streams
Functional interfaces
Optional
java.time
CompletableFuture
Default methods
```

Java 11

```
HTTP Client
String improvements
Files.readString/writeString
var in lambda parameters
single-file source execution
runtime/platform improvements
```

Java 17

```
Records
Sealed classes
Pattern matching for instanceof
modern language syntax/features
stronger JDK encapsulation
```

34. Java 8 Example in a Backend Project

Suppose you retrieve payments:

```
List<Payment> payments =
    repository.findAll();
```

You need total paid amount:

```
BigDecimal total =
    payments.stream()
        .filter(p -> "PAID".equals(p.getStatus()))
        .map(Payment::getAmount)
        .reduce(BigDecimal.ZERO,
            BigDecimal::add);
```

Here you are using:

```
Java 8
├─ Stream
├─ Lambda
├─ Method reference
└─ Functional programming
```

35. Java 11 Example

Suppose you need to call an external service:

```
HttpClient client =
    HttpClient.newHttpClient();

HttpRequest request =
    HttpRequest.newBuilder()
        .uri(URI.create(url))
        .GET()
```

```
        .build();

CompletableFuture<HttpResponse<String>> future =
    client.sendAsync(
        request,
        HttpResponse.BodyHandlers.ofString());
```

Here:

```
Java 11
→ standard HttpClient

Java 8
→ CompletableFuture
```

This shows that Java versions build on each other.

36. Java 17 Example

Modern DTO:

```
public record PaymentResponse(
    Long id,
    String status,
    BigDecimal amount) {
}
```

Then:

```
if (result instanceof PaymentResponse response) {
    System.out.println(response.status());
}
```

This is concise modern Java that you'll increasingly see in Java 17 Spring Boot codebases.

37. What Should You Actually Study?

For your target backend role, I would prioritize:

Java 8 — Must know deeply

```
***** Lambda
***** Streams
***** Functional Interfaces
***** Optional
***** Method References
***** Collections + Streams
***** CompletableFuture
***** java.time
```

Java 11 — Know practically

```
***** HttpClient
***** String API
***** Files API
***** var in lambda
```

Java 17 — Must know

```
***** Records
***** Sealed Classes
***** Pattern matching instanceof
***** Text Blocks
***** Java 17 migration implications
```

38. Common Interview Questions

Java 8

- What are lambda expressions?
- What is a functional interface?
- Predicate vs Function vs Consumer vs Supplier?
- `map()` vs `flatMap()` ?
- Intermediate vs terminal stream operations?
- Why are streams lazy?
- `orElse()` vs `orElseGet()` ?
- `findFirst()` vs `findAny()` ?
- Sequential vs parallel streams?
- What problem does `Optional` solve?
- What is `CompletableFuture` ?

Java 11

- What was introduced in Java 11?
- How does the Java 11 HTTP Client differ from `HttpURLConnection` ?
- What is `String.isBlank()` ?
- `strip()` vs `trim()` ?
- What is `Files.readString()` ?
- What is `var` in lambda parameters?

Java 17

- What are records?
- Can a record extend another class?
- Can a record implement an interface?
- Should JPA entities be records?
- What are sealed classes?
- What does `permits` do?
- What is pattern matching for `instanceof` ?
- What Java 17 features have you used in Spring Boot?

39. One Important Versioning Point

Don't say:

```
"Java 17 introduced every feature I've seen in a Java 17 project."
```

Some features commonly used in Java 17 applications were actually introduced earlier.

For example:

```
var                → Java 10
switch expressions  → Java 14
```

```
text blocks      → Java 15
records          → Java 16
sealed classes   → Java 17
```

This distinction can impress an interviewer because it shows you understand the evolution rather than memorizing a release checklist.

40. BPS/Project-Specific Interview Questions

Now inspect your actual codebase for the Java version features you can honestly claim.

Java 8

Search for:

```
stream()
filter()
map()
flatMap()
Optional
Predicate
Function
Consumer
Supplier
CompletableFuture
```

Then identify one real example where Java 8 reduced boilerplate or made processing clearer.

Java 11

Search for:

```
HttpClient
isBlank()
strip()
readString()
writeString()
```

Since your project uses synchronous HTTP communication to external payment services, inspect what HTTP client/library is actually used before claiming Java 11 `HttpClient`; your BPS context currently identifies `RestTemplate`, not Java's standard `HttpClient`.

Java 17

Search for:

```
record
sealed
permits
instanceof
```

Then ask yourself:

"If my project is running Java 17, which Java 17 features have I actually used in production code, and why?"

That's the best way to answer the common interview question:

"Which Java 8/11/17 features have you personally used in your project?"